



K.R. Mangalam University
School of Engineering and Technology

Task -1 Record execution time, memory usage, and operation counts.

Design & Analysis of Algorithms

Course Code - ENCA351

(2026-27)

Submitted by:

Diksha Goel
2401201173
BCA (AI & DS) - A

Submitted To:

Dr. Aarti Sangwan
Faculty, SOET

1. Objective

Implement an Automated Benchmarking Engine that executes algorithms and code snippets for multiple input sizes and records execution time, memory usage and operation counts. The engine is written in Python 3 (file: benchmark_engine.py) and is fully automated: one command runs every suite, prints a live log and saves the results to CSV and JSON.

2. Design of the engine

Component	Purpose / technique
OpCounter	Lightweight object passed into every algorithm. The algorithm increments counter.ops at each basic operation (comparison, multiplication, loop iteration) and counter.max_depth tracks recursion depth.
Algorithms under test	Linear, binary and jump search; iterative, recursive, memoized and built-in factorial; single, nested, dependent, triple and logarithmic loops (12 in total).
BenchmarkEngine.run()	For each input size: builds the input, measures time, measures memory, counts operations and stores a Record.
Time measurement	time.perf_counter(). Each sample is repeated 7 times and the median is reported (min and standard deviation are also stored). Very fast calls are batched (inner loop) so the timer resolution does not dominate.
Memory measurement	tracemalloc: peak bytes allocated during one call minus the baseline. Run as a separate pass because tracing slows execution.
Operation counts	Instrumented counter in a separate pass, so counting does not affect the memory pass.
Persistence	save() writes results.csv and results.json for later analysis and plotting.

3. Test configuration

Suite	Algorithm	Input sizes tested	Input type
Search	Linear / Binary / Jump	1,000; 5,000; 10,000; 50,000; 100,000; 500,000; 1,000,000	Sorted list, target absent (worst case)
Factorial	Iterative / Recursive / Memoized / Built-in	50; 100; 200; 400; 600; 800; 1000; 1500; 2000	Integer n
Loops	Single	1,000 – 500,000	Integer n
Loops	Nested, Dependent	50 – 1,000	Integer n
Loops	Triple	10 – 120	Integer n
Loops	Logarithmic	10 ³ – 10 ¹⁸	Integer n

4. Sample output (console)

Running python benchmark_engine.py produces a live log for every algorithm and input size (excerpt below; 91 measurements in total):

```

$ python benchmark_engine.py

== Search algorithms (worst case) ==

[search ] Linear Search      n=   1,000 time=  0.0228 ms mem=   0.16 KB ops=    1,000
[search ] Linear Search      n=   5,000 time=  0.1266 ms mem=   0.16 KB ops=    5,000
[search ] Linear Search      n=  10,000 time=  0.2479 ms mem=   0.16 KB ops=   10,000
[search ] Linear Search      n=  50,000 time=  1.2123 ms mem=   0.16 KB ops=   50,000
[search ] Linear Search      n= 100,000 time=  2.4250 ms mem=   0.16 KB ops=  100,000
[search ] Linear Search      n= 500,000 time= 12.1897 ms mem=   0.16 KB ops= 500,000
[search ] Linear Search      n=1,000,000 time= 24.9113 ms mem=   0.16 KB ops=1,000,000
[search ] Binary Search      n=   1,000 time=  0.0017 ms mem=   0.15 KB ops=         9
[search ] Binary Search      n=   5,000 time=  0.0022 ms mem=   0.18 KB ops=        12
[search ] Binary Search      n=  10,000 time=  0.0023 ms mem=   0.18 KB ops=        13
[search ] Binary Search      n=  50,000 time=  0.0026 ms mem=   0.18 KB ops=        15
[search ] Binary Search      n= 100,000 time=  0.0027 ms mem=   0.18 KB ops=        16
[search ] Binary Search      n= 500,000 time=  0.0030 ms mem=   0.18 KB ops=        18
[search ] Binary Search      n=1,000,000 time=  0.0031 ms mem=   0.18 KB ops=        19
[search ] Jump Search        n=   1,000 time=  0.0029 ms mem=   0.17 KB ops=        31
[search ] Jump Search        n=   5,000 time=  0.0045 ms mem=   0.17 KB ops=       70
[search ] Jump Search        n=  10,000 time=  0.0059 ms mem=   0.17 KB ops=      100
[search ] Jump Search        n=  50,000 time=  0.0114 ms mem=   0.17 KB ops=     223
[search ] Jump Search        n= 100,000 time=  0.0177 ms mem=   0.26 KB ops=     316
[search ] Jump Search        n= 500,000 time=  0.0455 ms mem=   0.26 KB ops=    707
[search ] Jump Search        n=1,000,000 time=  0.0697 ms mem=   0.26 KB ops=   1,000

== Factorial implementations ==

[factorial] Iterative        n=     50 time=  0.0042 ms mem=   0.20 KB ops=        49
[factorial] Iterative        n=    100 time=  0.0087 ms mem=   0.29 KB ops=       99
[factorial] Iterative        n=    200 time=  0.0199 ms mem=   0.48 KB ops=      199

```

Figure 1: Console output of the benchmarking engine (excerpt).

5. Summary of recorded measurements (largest input of each algorithm)

Algorithm	n	Time (ms)	Memory (KB)	Operations	Max depth
Linear Search	1,000,000	24.9113	0.16	1,000,000	0
Binary Search	1,000,000	0.0031	0.18	19	0
Jump Search	1,000,000	0.0697	0.26	1,000	0
Iterative	2,000	0.5665	5.17	1,999	0
Recursive	2,000	1.0148	55.11	1,999	2000
Memoized Recursive	2,000	1.2492	2,477.52	1,999	2000
Built-in math.factorial	2,000	0.1029	13.18	1	0
Single Loop	500,000	22.7978	0.23	500,000	0
Nested Loop (n^2)	1,000	40.3517	0.34	1,000,000	0
Dependent Loop (n^2/2)	1,000	18.9793	0.34	499,500	0
Triple Loop (n^3)	120	76.2457	0.28	1,728,000	0
Logarithmic Loop	1,000,000,000,000,000,000	0.0034	0.16	60	0

Appendix: complete source code (benchmark_engine.py)

```
"""
Lab 2: Algorithm Performance Measurement and Benchmarking Tool
=====

An automated benchmarking engine that runs algorithms / code snippets for multiple
input sizes and records:

*      execution time      (time.perf_counter, median of several repeats)
*      memory usage       (tracemalloc peak allocation during the call)
*      operation counts    (instrumented comparisons / iterations / calls)      * recursion depth
(extra "space" indicator for recursive code)

Results are written to CSV + JSON and later turned into charts / tables.
Usage:      python benchmark_engine.py          # runs every suite, saves
results/ ""
    import csv
    import json
    import os
    import statistics
    import sys
    import time
    import tracemalloc
    from dataclasses import dataclass, field, asdict
    from typing import Callable, Dict, List

sys.setrecursionlimit(10_000)

# -----
# 1. Instrumentation: operation counter
# ----- class
OpCounter:
    """Counts basic operations (comparisons, iterations, calls)."""
    __slots__ = ("ops", "depth", "max_depth")

    def __init__(self):
        self.ops = 0
        self.depth = 0
        self.max_depth = 0

# -----
# 2. Algorithms under test
# -----
# ---- Search algorithms (worst case: target is absent) ----
def linear_search(arr, target, c: OpCounter):
    for x in arr:
        c.ops += 1
        # one comparison per
        element if x == target:
            return True
    return False

def binary_search(arr, target, c: OpCounter):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        c.ops += 1
        # one comparison per halving step
        if arr[mid] == target:
            return True
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False

def jump_search(arr, target, c: OpCounter):
```

```

    n = len(arr)
    step = int(math.sqrt(n))    prev = 0    while prev < n
and arr[min(prev + step, n) - 1] < target:        c.ops += 1
prev += step    for i in range(prev, min(prev + step, n)):
c.ops += 1        if arr[i] == target:
    return True
return False

# ---- Factorial implementations ----- def
fact_iterative(n, c: OpCounter):
    result = 1    for i in
range(2, n + 1):
        c.ops += 1        # one multiplication per
iteration    result *= i    return result

def fact_recursive(n, c: OpCounter):
c.depth += 1
    c.max_depth = max(c.max_depth, c.depth)
if n <= 1:
    c.depth -= 1
return 1
    c.ops += 1        # one multiplication per
call    r = n * fact_recursive(n - 1, c)    c.depth -= 1    return
r

def fact_memoized(n, c: OpCounter, _cache=None):
if _cache is None:
    _cache = {0: 1, 1: 1}        # fresh cache for every top-level call
c.depth += 1
    c.max_depth = max(c.max_depth, c.depth)
if n in _cache:
    c.depth -= 1
return _cache[n]    c.ops
+= 1
    _cache[n] = n * fact_memoized(n - 1, c, _cache)
c.depth -= 1    return _cache[n]

def fact_builtin(n, c: OpCounter):
    c.ops += 1        # one call into optimised C code
return math.factorial(n)

# ---- Loop-based programs ----- def
loop_single(n, c: OpCounter):
    total = 0    for
i in range(n):
c.ops += 1
    total += i    return
total

def loop_nested(n, c: OpCounter):
    total = 0    for i in
range(n):        for j in
range(n):
c.ops += 1
    total += 1    return
total

def loop_dependent(n, c: OpCounter):
    total = 0    for i in
range(n):        for j in
range(i):

```

```

        c.ops += 1
total += 1    return total

def loop_triple(n, c: OpCounter):
    total = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                c.ops += 1
    total += 1    return total

def loop_logarithmic(n, c: OpCounter):
    i, total = 1, 0
    while i < n:
        c.ops += 1
    total += i
    *= 2    return total
total

# -----
# 3. Benchmark engine
# -----

@dataclass class
Record:      suite: str
algorithm: str    n:
int    time_ms: float
time_min_ms: float
time_std_ms: float
memory_kb: float
operations: int
max_depth: int

@dataclass class
BenchmarkEngine:
repeats: int = 7
    records: List[Record] = field(default_factory=list)

    def _measure_time(self, call: Callable[[], None]) -> List[float]:
        """Median-of-N timing; fast calls are batched for timer accuracy."""
        t0 = time.perf_counter()
        call()
        single = max(time.perf_counter() - t0, 1e-9)
        inner = max(1, int(2e-4 / single))    # aim for >= ~0.2 ms per
        sample    samples = []
        for _ in range(self.repeats):
            s = time.perf_counter()
            for _ in range(inner):
                call()
            samples.append((time.perf_counter() - s) / inner)
        return samples

    def run(self, suite: str, name: str, fn: Callable, sizes: List[int],
        build_input: Callable[[int], tuple]):
        """Benchmark `fn` for every input size in `sizes`."""
        for n in sizes:
            args = build_input(n)

            # --- time -----
            samples = self._measure_time(lambda: fn(*args, OpCounter()))

            # --- memory (separate pass; tracing slows execution) -----
            tracemalloc.start()
            tracemalloc.reset_peak()
            base, _ = tracemalloc.get_traced_memory()
            fn(*args, OpCounter())
            _, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()

```

```

# --- operation count -----
-      c = OpCounter()          fn(*args, c)

        self.records.append(Record(
suite=suite, algorithm=name, n=n,
time_ms=statistics.median(samples) * 1e3,
time_min_ms=min(samples) * 1e3,
        time_std_ms=(statistics.pstdev(samples) * 1e3),
memory_kb=max(peak - base, 0) / 1024,
operations=c.ops,          max_depth=c.max_depth,
        ))
        r = self.records[-1]
        print(f"    [{suite:9s}] {name:22s} n={n:>9,d} "
              f"time={r.time_ms:10.4f} ms  mem={r.memory_kb:10.2f} KB "
f"ops={r.operations:>12,d}")

# ---- persistence -----
def save(self, folder: str):          os.makedirs(folder, exist_ok=True)
rows = [asdict(r) for r in self.records]      with
open(os.path.join(folder, "results.json"), "w") as f:
    json.dump(rows, f, indent=2)      with open(os.path.join(folder,
"results.csv"), "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
w.writeheader()
        w.writerows(rows)

# -----
# 4. Benchmark configuration
# ----- def
sorted_input(n):
    return (list(range(n)), -1)          # absent target => worst case

def int_input(n):
    return (n,)

def run_all(out_dir="results"):
    eng = BenchmarkEngine(repeats=7)

    print("\n== Search algorithms (worst case) ==")          search sizes = [1 000,
5 000, 10 000, 50 000, 100 000, 500 000, 1 000 000]
    eng.run("search", "Linear
Search", linear search, search sizes, sorted_input)
    eng.run("search", "Binary
Search", binary search, search sizes, sorted_input)
    eng.run("search", "Jump
Search", jump_search, search_sizes, sorted_input)

    print("\n== Factorial implementations ==")
    fact_sizes = [50, 100, 200, 400, 600, 800, 1000, 1500, 2000]
    eng.run("factorial", "Iterative", fact_iterative, fact_sizes, int_input)
    eng.run("factorial", "Recursive", fact_recursive, fact_sizes, int_input)
    eng.run("factorial", "Memoized Recursive", fact_memoized, fact_sizes, int_input)
    eng.run("factorial", "Built-in
math.factorial", fact_builtin, fact_sizes, int_input)

    print("\n== Loop-based programs ==")
    eng.run("loops", "Single Loop", loop_single,
[1 000, 5 000, 10 000, 50 000, 100 000, 500 000],
int_input)
    eng.run("loops", "Nested Loop (n^2)", loop_nested,
[50, 100, 200, 400, 600, 800, 1000], int_input)
    eng.run("loops", "Dependent Loop (n^2/2)", loop_dependent,
[50, 100, 200,
400, 600, 800, 1000], int_input)
    eng.run("loops", "Triple Loop
(n^3)", loop_triple,
[10, 20, 40, 60, 80, 100, 120],
int_input)
    eng.run("loops", "Logarithmic Loop", loop_logarithmic,
[10**3, 10**4, 10**5, 10**6, 10**8, 10**12, 10**18], int_input)

    eng.save(out_dir)          print(f"\nSaved {len(eng.records)}
records to '{out_dir}/'")      return eng

if __name__ == "__main__":

```

```
run_all()
```